

National University of Computer and Emerging Sciences

Artificial Intelligence

(AL-2002)

Lab Mid Exam

Paper A

Date: March 18th, 2025

Course Instructor(s)

Mr. Talha Shahid, Miss. Alishba Subhani

Total Time (hr): 1.5

Total Points: 50

Total Questions: 03

Student Name

Roll No

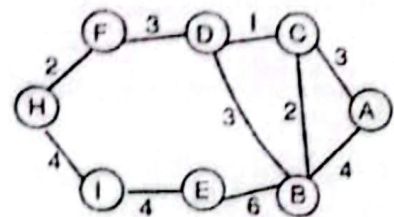
Section

Student Signature

Question # 01 (LLO: 1)

[18 Points, 8 Weightage]

In a densely connected city represented as a weighted graph, intersections are nodes and roads have varying lengths as edges. An agent perceives its environment by detecting nearby intersections and roads, each with a specific cost (distance). The agent's goal is to find the least-cost path between two locations using **Uniform Cost Search (UCS)**.



Agent's Starting Point: A | Agent's Goal: Reach I

Tasks:

1. Define the architecture of a goal-based agent, and environment class. [07]
2. Implement the **Uniform Cost Search (UCS)** function within the agent class to find the least-cost path. [08]
3. Create a `run_agent()` function that simulates the agent in the specified environment and prints the path and total cost. [03]

Question # 02 (LLO: 3)

[18 Points, 9 Weightage]

In a **space exploration game**, a rover needs to navigate an alien planet's surface to find an energy module hidden at a specific location. The terrain is represented as a grid where:

- . represents a traversable path.
- # represents an obstacle.
- S represents the rover's starting point
- E represents the energy module's location.

The rover can move up, down, left, or right. Use **A Search*** with the **Manhattan distance** to the energy module as a heuristic and the actual movement cost between cells.

S	.	#	.	E
.	#	.	#	.
.	.	.	#	.
#	.	#	.	#
.	.	.	.	.

S = Rover Starting Point

E = Energy module's location

. = Traversable Path

= Obstacle

Spring 2025

FAST School of Computing

Page 1 of 2

National University of Computer and Emerging Sciences

Task:

1. Define the heuristic function which calculates the Manhattan distance between a point and the energy module. ($|x1 - x2| + |y1 - y2|$). [03]
2. Define a **generate_neighbours** function which provides all possible neighbours of a given state based on the possible moves of the rover. [04]
3. Implement the **A* function** which finds the shortest path from the start to the energy module, considering both the heuristic cost and the actual cost. [08]
4. A main function which calls the A* Search function and displays the shortest path taken by the rover, the total number of steps, and the total cost. [03]

Question # 03 (ILO: 4)

[14 Points, 7 Weightage]

Genetic Algorithm for the 0/1 Knapsack Problem [refer to the provided guide.

A logistics company is optimizing its delivery truck space by selecting the most valuable set of packages that fit within a weight limit. Each package has a specific weight and value, and the company needs to maximize total value while ensuring the total weight does not exceed the truck's capacity.

To solve this problem, you will implement a **Genetic Algorithm (GA)**. Some parts of the algorithm are already provided, and your task is to complete the missing functions.

You **must implement** the following functions to complete the GA:

1. **create_random_individual(num_items)**
 - o Generate a random binary list representing a possible solution (1 = item included, 0 = item excluded).
2. **initialize_population(pop_size, num_items)**
 - o Create an initial population of pop_size individuals using 'create_random_individual()'.
3. **select_parents(population, fitness_scores)**
 - o Select the top 50% of the population based on fitness for reproduction.
4. **crossover(parent1, parent2)**
 - o Implement **single point crossover**.